

```
# g++ regex1.cc
# ./a.out
Match Found
# g++ regex2.cc
# ./a.out
No Match Found
#
```

Figure 1: Output of `regex_search()` and `regex_match()`

just like the program `regex1.cc`. Since I am expecting a mixed group of readers with expertise in different programming languages, I tried to make the C++ programs look as much as possible like C programs, in the belief that C is the language of academia and everybody has had a stint with it as a student. I could have even used the `printf()` and `scanf()` functions instead of `cout` and `cin`. But a line should be drawn somewhere, and this is where I have stopped making C++ programs look like C.

The next line of code `regex_pat("Source");` is very important. It is responsible for setting up the regular expression pattern that should be searched in the string *Open Source For You*. Here the pattern searched is the word 'Source' which is stored in an object called `pat` of the class `regex`.

The next few lines of code contain an *if-else* statement. The line of code `if( regex_search(str,pat) )` uses the function `regex_search()` provided by the header file `<regex>` to search for the pattern stored in the object `pat` of the class `regex` in the string stored inside the character array `str[]`. If a match is found, the line of code `cout << "Match Found\n";` is executed and prints the message *Match Found*. If a match is not found, the else part of the code `cout << "No Match Found\n";` is executed and prints the message *No Match Found*. This program can be compiled with the command `g++ regex1.cc`, where `g++` is the C++ compiler provided by GCC (GNU Compiler Collection). This will produce an executable called `a.out`. This is then executed with the command `./a.out`. On execution, the program prints the message *Match Found* on the screen because the function `regex_search()` searches the entire string to find a match. Since the word *Source* is present in the string *Open Source For You*, a match is found.

Now, it is time for us to revisit the difference between the functions `regex_search()` and `regex_match()`. To do this, the line of code `if( regex_search(str,pat) )` in the program `regex1.cc` is replaced with the line `if( regex_match(str,pat) )`.

This modified code is available as a program named `regex2.cc`, which can be compiled with the command `g++ regex2.cc`, which will produce an executable called `a.out`. This is then executed with the command `./a.out`. Now the output printed on the screen is *No Match Found*. Why? As mentioned earlier, this is due to a difference between the functions `regex_search()` and `regex_match()`. The function `regex_search()` searches the entire string for a match and the function `regex_match()` returns true only if the regular expression pattern is present at the beginning of a string. In

this case, the word *Source* appears as the second word in the string *Open Source For You*, and hence no match is found by the function `regex_match()`. Figure 1 shows the output of the programs `regex1.cc` and `regex2.cc`.

Pattern replacement in C++

Let's now study the working of the function `regex_replace()`. Let's consider the program `regex3.cc` which uses the function `regex_replace()`. This function will search for a match and if it finds one, the function will replace the matched string with a replacement string.

```
#include <iostream>
#include <regex>
#include <string>

using namespace std;

int main( )
{
    char str[] = "Open Source Software is Good";
    regex pat("Open Source");
    char rep[] = "Free";
    cout << regex_replace(str,pat,rep)<<'\n';
    return 0;
}
```

Except for the line of code `cout << regex_replace(str,pat,rep)<<'\n';` I don't think any further explanation is required. This is the line in which the function `regex_replace()` is called with three parameters -- the character array `str[]` where the string to be searched is stored, the regular expression pattern to be matched stored in the object `pat` of the class `regex`, and the pattern to be replaced stored in the character array `rep[]`. Execute the program `regex3.cc` with the commands `g++ regex3.cc` and `./a.out`. You will see the message *Free Software is Good* on the screen. Nothing surprising there, because the string in the character array `str[]` contains *Open Source Software is Good*, the pattern to be searched is *Open Source* and the replacement pattern is *Free*. Hence, a match is found and a replacement is done by the function `regex_replace()`.

The next question to be answered is whether the function `regex_replace()` behaves like the function `regex_search()` or the function `regex_match()`. In order to understand the behaviour of the function `regex_replace()` clearly, let us modify the program `regex3.cc` slightly to get a program called `regex4.cc` as shown in the following code:

```
#include <iostream>
#include <regex>
#include <string>

using namespace std;
```